WASM and Python

THE FUTURE OF SERVERLESS COMPUTING

HI 👋

Farhaan Bukhsh

Senior Software Engineer OpenCraft
Open edX Core Contributor

@farhaanbukhsh

Kumar Anirudha

Solutions Architect, Product Consultant

@anistark

WEBASSEMBLY

Originally designed for browsers... now expanding into serverless and sandboxed environments

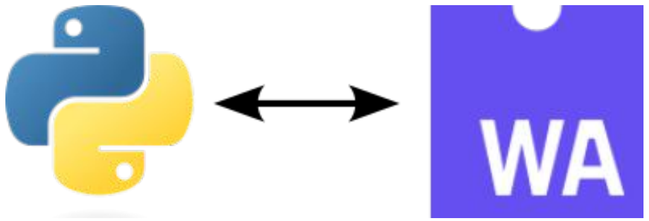
- Near-instant startup times.
- Enhanced security through sandboxing.
- Compact and platform-agnostic modules.
- Suitable for diverse environments, specially decentralised ecosystems.

CHALLENGES OF RUNNING PYTHON IN WASM

python’s dynamic nature

Unlike Rust and Go, Python's dynamic typing and runtime features pose challenges for efficient WASM execution.

- Performance overhead.
- Limited support for certain Python features and libraries.



Pyodide: CPython Compiled to WASM

Pyodide brings the Python runtime to the browser
by compiling CPython to WebAssembly


```python
import pyodide

async def run_python_code():
    py = await pyodide.loadPyodide()
    result = py.runPython("print('Hello from Pyodide!')")
    print(result)

await run_python_code()
```

PYODIDE NOT A TRANSPILER

Pyodide doesn’t convert python file to WASM rather:

- Brings the whole python VM in the browser.
- Libraries are also compiled with Python runtime.
- Python instructions run inside the Browser using JavaScript interface.

Benefits and Inspiration

PYODIDE WA has been a big inspiration in the wasm Python realm:

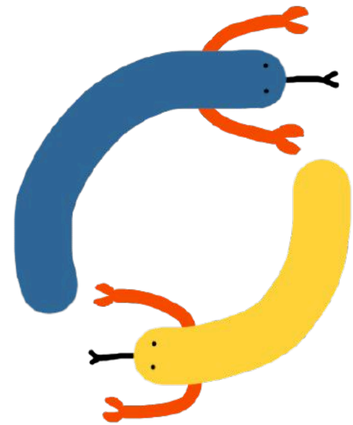
- Beautiful PEP by Hood PEP-776 [https://peps.python.org/pep-0776/]
- Patches for CPython and Emscripten is 🧠
- Persistence of the team to get support for Python in wasm

Downside of using Pyodide 🔻

- Huge size of the code being loaded in the browser
- Instead of the Python code into WASM conversion it takes a longer path.
- Too many patches and workarounds

EXPLORING CURRENT SOLUTIONS

RustPython: A Rust-based Python Interpreter



RustPython is a Python interpreter written in Rust, optimized for WASM.

```
git clone https://github.com/RustPython/RustPython.git
cd RustPython
cargo build --release --target wasm32-wasi
```

```
use rustpython_vm::Interpreter;

fn main() {
    let interpreter = Interpreter::default();
    interpreter.enter(|vm| {
        vm.run_code("print('Hello from RustPython!')", vm.ctx.new_scope()).unwrap();
    });
}
```

```
wasmtime rustpython.wasm
```

SO, WHERE DO WE STAND...

```
wrk -t4 -c100 -d30s http://localhost:8000/
```

Environment	Requests/sec	Avg Latency
WASM (Pyodide)	~500	10-30ms
Docker (FastAPI)	~2000+	3-5ms
WASM (RustPython)	~800	7-15ms
WASM (Rust)	~5000+	<1ms
WASM (Go)	~4000+	<2ms

A BETTER WAY?

Python to WASM

Feature	Python *on* WASM	Python *to* WASM
Interpreter needed?	✅ Yes	❌ No
Startup size	❌ Large (MBs)	✅ Tiny (KBs)
Runtime speed	❌ Slower (interpreted)	✅ Near-native
Language support	✅ Full Python	❌ Subset/static
Deployment	❌ Heavy	✅ Lightweight
Use case	Education, prototyping, data science	WASM microservices, games, tooling, embedded scripts

EXPLORING CURRENT SOLUTIONS

Py2wasm

- py2wasm is a compiler that transforms Python code into WebAssembly
- It leverages Nuitka, a Python-to-C compiler, to convert Python code into C, which is then compiled into Wasm.

$ pip install py2wasm

Naaah! 🤷

- It requires a very specific version of python i.e python 3.11.0
- Very difficult to configure and maintain
- Still the PR is open and fortunately maintained -- https://github.com/Nuitka/Nuitka/pull/2814

How Does py2wasm Work?

Python to C Compilation: py2wasm uses Nuitka to compile Python code into C code. Nuitka translates Python constructs into equivalent C representations, closely interfacing with the CPython API.

C to Wasm Compilation: The generated C code is then compiled into WebAssembly using a suitable C compiler that targets Wasm, such as Clang with WebAssembly support.

Execution in Wasm Runtimes: The resulting .wasm module can be executed in Wasm-compatible environments like Wasmer, allowing the Python application to run outside the traditional Python interpreter.

Solomon Hykes
@solomonstre

If WASM+WASI existed in 2008, we wouldn't have needed to created Docker. That's how important it is. Webassembly on the server is the future of computing. A standardized system interface was the missing link. Let's hope WASI is up to the task!

> 
> **Lin Clark** @linclark · Mar 27, 2019
>
> WebAssembly running outside the web has a huge future. And that future gets one giant leap closer today with...
>
> 📢 Announcing WASI: A system interface for running WebAssembly outside the web (and inside it too)...




INTRODUCING

wasmrun

A language-agnostic native WASM runtime

github.com/anistark/wasmrun

TAKEAWAYS

Python's integration with WASM is progressing but still lags behind languages like Rust and Go.

- The only proper way forward towards a serverless centric ecosystem is to develop the Python WASM ecosystem first.
- We need significant development in language support towards WASM native.
- We need a proper runtime and dev tools to push towards this goal.
- Community contributions and innovations will play a crucial role.

Thank you for your patience

Q & A

@kranirudha

Find us on 𝕏

@fhackdroid